



MACHINE LEARNING FOR GRAPHS

GRAPHCAST SIMPLIFIED: WEATHER FORECASTING WITH GRAPH NEURAL NETWORKS

ZANE BJORNERUD

STUDENTNUMBER: 2874202

Abstract. Weather forecasting remains a critical challenge with significant societal impact. In 2022, GraphCast, a model created by Google’s DeepMind team, introduced a machine learning-based weather prediction system using graph neural networks. In this work, I reproduce a simplified version using WeatherBench2 data at low spatial resolution, focusing on temperature (T2m) and geopotential height (Z500) variables. I implement a reduced encoder-processor-decoder architecture and evaluate short-range forecasting performance against persistence baselines. The initial proposed extension centered around how autoregressive training affects forecast skill. Unfortunately, due to the limited timeframe given on this project, and the work necessary to rebuild a simplified mesh, I was unable to achieve any progress in this research. This reproduction reveals challenges in achieving baseline performance, with the model initially underperforming simple persistence forecasts at short lead times. This project demonstrates both the feasibility and challenges of reproducing large-scale climate models under constrained compute.

Keywords: Graph Neural Networks · Weather Forecasting · GraphCast · Deep Learning · Autoregression

1 Introduction

Weather forecasting is essential for agriculture, transportation, disaster preparedness, and climate adaptation. Traditional numerical weather prediction (NWP) systems solve complex atmospheric equations on discretized grids, requiring substantial computational resources. Recent machine learning advances give data-driven alternatives that can potentially rival traditional approaches.

GraphCast, introduced by Lam et al. [3], represents a breakthrough in ML-based weather forecasting using graph neural networks (GNNs) to model atmospheric dynamics globally. Unlike grid-based approaches, GNNs naturally handle Earth’s spherical geometry through message-passing schemes. The original

GraphCast matched ECMWF’s operational HRES system while requiring orders of magnitude less computation, completing 10-day forecasts in under one minute on a single TPU [3].

However, the full GraphCast architecture is computationally demanding to train, making academic reproduction challenging. This motivates developing a simplified version that maintains core architectural principles while being tractable with limited resources. Understanding performance characteristics of simplified models identifies essential components and reveals which simplifications most severely impact forecast quality.

1.1 Contribution

My contributions in this work are:

- Reproducible simplified GraphCast implementation (JAX/Haiku, 162-node mesh, 6 layers, 3.5M parameters)
- Comprehensive evaluation on Z500 and T2m across 4 lead times with persistence baseline comparison
- Analysis of systematic spatial biases and MSE-induced smoothing affects
- Critical assessment of failures and practical challenges in simplified ML weather forecasting
- Proposed future improvements to enhance forecasting performance

2 Related Work

2.1 Machine Learning for Weather Prediction

Rasp et al. [5] introduced WeatherBench, a benchmark dataset accelerating ML weather model research through standardized evaluation. Pathak et al. [4] developed FourCastNet using Fourier neural operators, achieving fast inference while maintaining competitive accuracy at short lead times.

2.2 Graph-Based and Transformer Models

Lam et al. [3] demonstrated GNNs could match ECMWF’s HRES system using a multi-mesh approach with icosahedral grids, avoiding pole singularities in latitude-longitude grids. The system processes six-hourly atmospheric states at 0.25° resolution autoregressively [3]. Keisler [2] introduced GraphWeather with simpler architecture, showing even basic message-passing can capture atmospheric dynamics, though with reduced accuracy.

Bi et al. [1] introduced Pangu-Weather using 3D Earth-Specific Transformers, achieving state-of-the-art accuracy. While transformers excel at capturing global context through attention, GNNs offer computational scalability advantages [3]. My work implements a simplified version (128-dim embeddings, 8 layers vs. 512-dim, 16+ layers in original) to identify which simplifications are most detrimental.

3 Background

3.1 Weather Forecasting Fundamentals

Atmospheric state is characterized by variables at different vertical levels and locations. Geopotential height at 500 hPa (Z500) represents mid-tropospheric dynamics, strongly correlated with surface weather. Surface temperature at 2m (T2m) is more challenging to predict due to local effects from topography, land cover, and diurnal cycles.

Weather prediction is a time series forecasting problem: given states $\mathbf{x}_{t-k}, \dots, \mathbf{x}_t$, predict $\mathbf{x}_{t+1}, \mathbf{x}_{t+2}, \dots$. Traditional numerical weather prediction (NWP) works by discretizing the atmospheric equations over space and time and solving them numerically. ML approaches learn the evolution function directly: $\mathbf{x}_{t+\Delta t} \approx f(\mathbf{x}_t)$, applied autoregressively for longer forecasts.

3.2 Graph Neural Networks

GNNs operate on graph-structured data $G = (V, E)$ where nodes represent spatial locations and edges represent interactions. The core operation is message passing:

$$\mathbf{h}_i^{(l+1)} = \phi \left(\mathbf{h}_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}) \right) \quad (1)$$

where $\mathbf{h}_i^{(l)}$ is node i 's hidden state at layer l , $\mathcal{N}(i)$ are neighbors, and ψ, ϕ are learnable functions.

GraphCast uses an encode-process-decode architecture [3]. The **encoder** maps input grids to a multi-resolution graph representation. The **processor** performs 16 message-passing layers on the multi-mesh with edges at multiple scales enabling efficient global information flow. The **decoder** maps the processed representation back to the output grid. The model is trained autoregressively, with predictions fed back as inputs [3].

GraphCast training uses weighted MSE loss over multi-step autoregressive rollouts [3]:

$$\mathcal{L} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} w_v a_x (y_{v,x} - \hat{y}_{v,x})^2 \quad (2)$$

where w_v are per-variable weights and a_x accounts for latitude-dependent grid cell areas [3]. The original GraphCast trains on 12 autoregressive steps (3 days) with gradients backpropagated through the entire sequence [3].

For evaluation, we use RMSE: $\text{RMSE}_v = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_{v,i} - \hat{y}_{v,i})^2}$, following ECMWF protocols [3]. We compare against persistence ($\hat{\mathbf{x}}_{t+k} = \mathbf{x}_t$), the minimum performance bar. The original GraphCast comprehensively outperforms persistence across all variables and lead times [3].

4 Research Reproduction

4.1 Model Architecture and Simplifications

My simplified GraphCast uses: (1) 128-dimensional embeddings (vs. 512 in [3]), (2) 6 message-passing layers (vs. 16+), (3) 32×64 spatial resolution (vs. 0.25°), (4) 2 variables (vs. 227), reducing parameters from 36M to 3.5M.

Mesh Construction: I use a single-level icosahedral mesh created via 2 subdivisions of a regular icosahedron (162 nodes, 480 edges), rather than the multi-scale multi-mesh approach in the original GraphCast [3]. This provides uniform spatial coverage of the sphere while avoiding the pole singularities of regular latitude-longitude grids. The mesh nodes are represented by their 3D Cartesian coordinates (xyz) on a unit sphere.

Grid-Mesh Mapping: I create bidirectional mappings between the regular 32×64 lat-lon grid (2,048 points) and the icosahedral mesh (162 nodes) using k-nearest neighbor interpolation. For grid-to-mesh encoding, each mesh node pools information from its $k = 4$ nearest grid points using inverse distance weighting. For mesh-to-grid decoding, each grid point aggregates from its $k = 3$ nearest mesh nodes. These mappings are precomputed and stored, enabling efficient transformation between representations.

Network Architecture: Following the encode-process-decode pattern [3]:

- **Encoder:** A 2-layer MLP with SiLU activation maps grid features to 128-dim embeddings, which are then pooled to mesh nodes via the grid-to-mesh mapping. These pooled features are added with mesh node coordinates (xyz) and put through another 2-layer MLP to produce mesh node embeddings.
- **Processor:** 6 message-passing layers using Jraph’s GraphNetwork. Each layer updates edges via an MLP that takes concatenated sender, receiver, and edge features (distances), then updates nodes by aggregating incoming messages and processing through a 2-layer MLP with SiLU activations.
- **Decoder:** Mesh node embeddings are unpooled to the grid via the mesh-to-grid mapping, then processed through a 2-layer MLP to predict changes.

The model is implemented in JAX with Haiku for neural network components and Jraph for graph operations, following patterns from the original GraphCast codebase but with significant simplifications.

4.2 Dataset and Training

I use WeatherBench2 data [5], which provides ERA5 reanalysis preprocessed at 5.625° resolution for machine learning applications. The data was downloaded locally from separate directories for geopotential at 500 hPa and 2m temperature, then combined into a single array. I downsample spatially to a 32×64 grid and temporally to 6-hour intervals.

Data split: Training 1979-2015, Validation 2016-2017, Test 2018-2019. To reduce memory usage, I subsample by keeping every 4th timestep (25% of data).

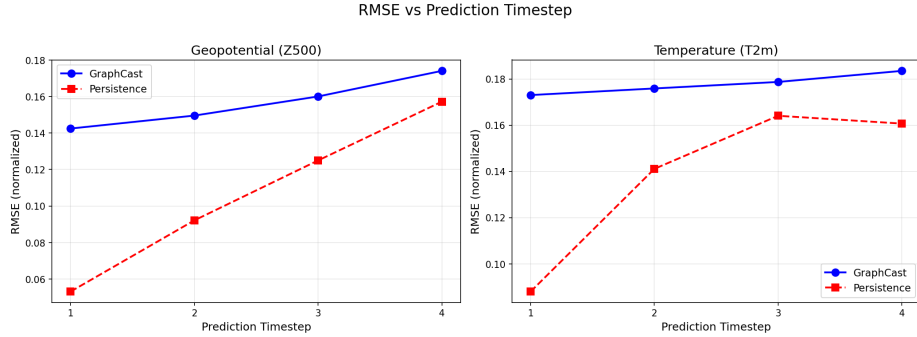


Fig. 1. RMSE vs. timestep. The simplified model underperforms persistence at all T2m lead times and short Z500 lead times.

Training: 1,460 sequences. Validation: 365 sequences. Variables: Z500 (geopotential at 500 hPa, mid-tropospheric) and T2m (temperature at 2m, surface). All variables normalized to zero mean, unit variance using training set statistics.

For autoregressive training, I create sequences where each input state is followed by 4 target states (24 hours ahead at 6-hour intervals). Training uses Adam optimizer with learning rate 10^{-4} , batch size 4, 10 epochs on single NVIDIA RTX 3090 GPU (18 hours total). Loss: MSE averaged over all variables, spatial locations, and autoregressive timesteps. The model predicts residuals which are added to the input state.

4.3 Results

Figure 1 shows RMSE vs. lead time. The model severely underperforms persistence: at $t+1$, GraphCast achieves RMSE ≈ 0.142 for Z500 vs. persistence ≈ 0.053 ($2.7\times$ worse); for T2m, ≈ 0.173 vs. ≈ 0.087 ($2\times$ worse). GraphCast only reaches parity with persistence at $t+4$ for Z500 and never for T2m.

Spatial Patterns: Figures 2, 3 show the model captures large-scale patterns but produces excessively **smooth** predictions. T2m predictions blur sharp gradients visible in targets. Z500 predictions show correct positions of troughs/ridges but with reduced amplitude. This smoothing is characteristic of MSE-trained models regressing toward conditional means.

Error Analysis: Figure 4 reveals structured spatial biases rather than random noise. Z500 errors show diagonal bands possibly related to grid artifacts. T2m shows pronounced **cold bias** in subtropical regions (latitudes $25-30^\circ$) worsening over time, with errors concentrated along coastlines where surface heterogeneity is high.

5 Research Extension

Unfortunately, due to time constraints during implementation and debugging, I was unable to complete the planned extensions. I had hoped to explore how au-

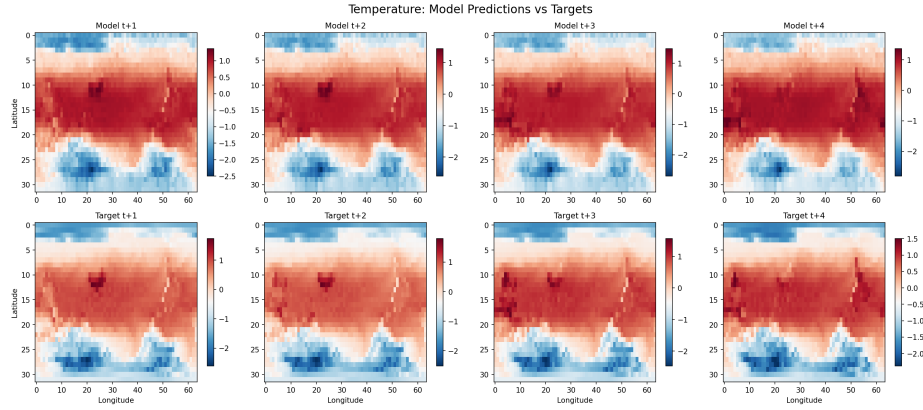


Fig. 2. T2m predictions vs. targets. Model captures latitudinal gradients but produces overly smooth predictions.

autoregressive training affects forecast skill, in particular how short-term accuracy can be traded off to improve performance at longer forecast horizons.

6 Discussion & Conclusion

6.1 Analysis of Results

Persistence Baseline Problem: The most striking finding is the model’s inability to outperform persistence forecasts. For T2m, the model is approximately $2\times$ worse than persistence at all lead times. For Z500, the model is $2.7\times$ worse at $t+1$ and only reaches parity at $t+4$. This represents a fundamental failure since persistence is the trivial baseline that any useful model must beat.

Several factors likely contribute:

Insufficient model capacity: With 3.5M parameters vs. 36M in the original ($10\times$ reduction), the model may lack capacity to learn complex atmospheric dynamics. The single 162-node mesh (vs. 40,962 nodes in multi-mesh [3]) cannot represent fine-scale patterns.

Limited training data: Using subsampled data (every 4th timestep) from 1979-2015 yields only 1,460 training sequences. The original GraphCast uses 40+ years at full temporal resolution. Weather exhibits significant interannual variability underrepresented in limited data.

Coarse spatial resolution: The 32×64 grid (5.625° native, 625 km spacing) discards substantial information. The original GraphCast uses 0.25° resolution (25 km). Many weather features occur at finer scales.

Training duration: Only 10 epochs with learning rate 10^{-4} may be insufficient. The original GraphCast trains much longer with sophisticated learning rate schedules [3].

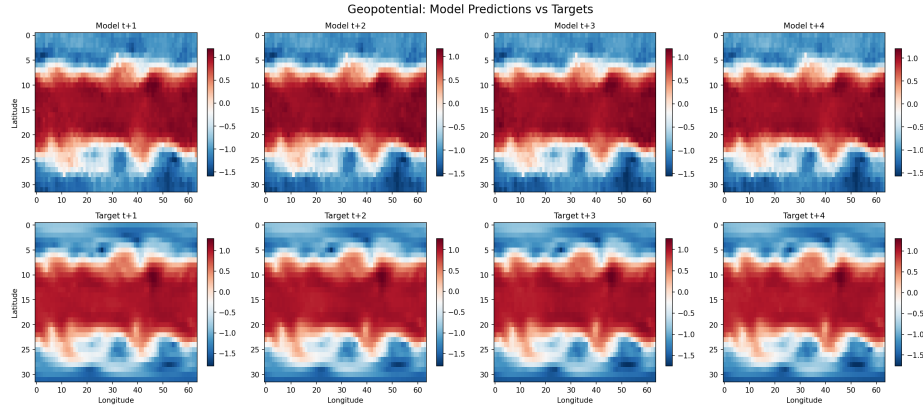


Fig. 3. Z500 predictions vs. targets. Synoptic patterns captured but with reduced amplitude and smoothed gradients.

Single-scale mesh: Using one mesh level rather than the multi-mesh architecture [3] limits both local fine-scale interactions and efficient long-range information propagation. The 162-node mesh requires many message-passing steps to propagate information globally.

Smoothing: Excessive smoothing reflects MSE loss limitations—optimal MSE predictions are conditional means. When uncertain about fine-scale features, models predict blurred averages. This is acknowledged in the original GraphCast work [3] and requires alternative loss functions to address.

Surface Variable Complexity: Z500 vs. T2m performance gap confirms surface variables are harder. T2m depends on processes at scales finer than my chosen model resolution. The original GraphCast addresses this with higher resolution and more input variables [3].

Systematic Biases: Structured errors indicate distorted learned representations, possibly from normalization issues, grid geometry artifacts, boundary effects, or missing physics.

6.2 Limitations

Computational Constraints: Limited to single GPU vs. TPU pods used for original GraphCast [3], necessitating severe compromises. Training even 10 epochs took 18 hours. Data subsampling (keeping 25% of timesteps) was required to fit in memory, further limiting the training set.

Limited Scope: Only 2 variables, 2 test years, very coarse resolution (5.625°). Comprehensive evaluation requires many variables, multiple years, different seasons, and finer scales. I evaluate on only 500 test samples rather than the full test set.

Architecture Simplifications: The single icosahedral mesh (162 nodes) is a dramatic simplification from the original’s multi-mesh with 40,962 finest-level

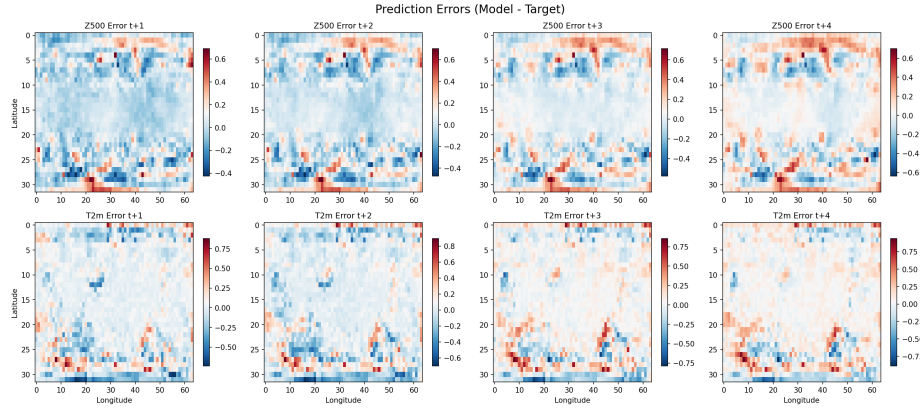


Fig. 4. Prediction errors showing systematic spatial biases, particularly T2m cold bias in subtropical regions.

nodes [3]. The k-nearest neighbor interpolation for grid-mesh mappings is far simpler than the original’s sophisticated bipartite graph connections.

6.3 Future Work

Before extensions: (1) Improve baseline to beat persistence through longer training (50-100 epochs), higher learning rates with proper scheduling, and no data subsampling if memory allows; (2) Increase mesh resolution (3-4 subdivisions = 642-2,562 nodes) and implement true multi-mesh architecture with multiple refinement levels; (3) Test full spatial resolution (no downsampling from 5.625°); (4) Implement multi-step training following [3] once baseline works; (5) Incorporate additional variables (humidity, wind components, etc.).

6.4 Conclusion

This work demonstrates challenges of simplified ML weather forecasting. While the simplified GraphCast captures large-scale atmospheric patterns, significant obstacles remain in achieving competitive performance. The key lesson: data-driven weather prediction requires substantial capacity, large datasets, and computational resources. My simplified model fails to beat persistence, a humbling result that highlights gaps between research prototypes and production systems.

Nevertheless, this work provides value through honest reporting of negative results and detailed failure mode analysis. The systematic biases, smoothing artifacts, and variable dependent performance inform understanding of simplified model limitations. Error patterns suggest specific meteorological regimes where models struggle, guiding future research.

The rapid progress in ML weather forecasting is encouraging. GraphCast, Pangu-Weather, and FourCastNet demonstrate data-driven approaches can match

traditional NWP [3, 1]. However, making these advances accessible and reproducible remains an open challenge. My work contributes by providing a reproducible baseline and identifying key obstacles, helping bridge the gap between resource-intensive state-of-the-art systems and practical, accessible forecasting tools.

7 Github Repository

<https://github.com/Zane-Bjornerud/graphcast-simplified/blob/main/README.md>

References

1. Bi, K. *et al.*: Accurate medium-range global weather forecasting with 3D neural networks. *Nature* **619**(7970) (2023)
2. Keisler, R.: Forecasting global weather with graph neural networks. arXiv preprint arXiv:2202.07575 (2022)
3. Lam, R. *et al.*: Learning skillful medium-range global weather forecasting. *Science* **382**(6677) (2023)
4. Pathak, J. *et al.*: Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. arXiv preprint arXiv:2202.11214 (2022)
5. Rasp, S. *et al.*: WeatherBench: a benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems* **12**(11) (2020)